



LOST BOYS CYBER

THREAT INTELLIGENCE • MALWARE ANALYSIS • DETECTION ENGINEERING



CVE-2022-0492 Linux Kernel Improper Authentication Vulnerability

CVE / Vulnerability Threat Intelligence Report

Classification	TLP:CLEAR
Published	2026-06-02
Version	1.1
Author	Lost Boys Cyber
Report Type	CVE / Vulnerability Threat Intelligence Report

TLP:CLEAR | Lost Boys Cyber | CVE-2022-0492 Linux Kernel Improper Authentication Vulnerability - CVE / Vulnerability Threat Intelligence Report

Published: 2026-06-02 | TLP:CLEAR | Version: 1.1 | Author: Lost Boys Cyber

AI Generation: This report was generated with AI assistance and is provided for informational purposes only. All findings, IOCs, detection rules, hunting queries, attribution assessments, and recommendations must be independently reviewed and validated by a qualified security professional before operational use.

Table of Contents

1. Executive Summary
2. Vulnerability Metadata
3. Source Review & Confidence
4. Vulnerability Details
5. Attack Scenarios
6. Threat Landscape & Exploitation Status
7. Affected Products and Exposure
8. Mitigation and Workarounds
9. Detection Engineering
10. Threat Hunting
11. MITRE ATT&CK Mapping
12. Validation / Lab Testing
13. Recommended Actions Summary
14. References

CVE-2022-0492 Linux Kernel Improper Authentication Vulnerability

1. Executive Summary

CVE-2022-0492 is a critical Linux kernel vulnerability associated with the `cgroup\_release\_agent\_write` logic in cgroup v1. Public reporting and the NVD describe the flaw as enabling unauthorized users to modify the `release\_agent` file at the root of the cgroup hierarchy. In practical terms, this can be abused for local privilege escalation and, in containerized environments, container escape to host-level code execution when preconditions are met.

This issue remains operationally important because vulnerable Linux hosts and container platforms can expose a high-impact privilege boundary failure. Environments relying on cgroup v1, unprivileged container workloads, or weak hardening around namespace and capability boundaries should treat this issue as both a host-hardening and container-security priority.

2. Vulnerability Metadata

Field	Value
CVE	CVE-2022-0492
Platform	Linux kernel
Component	`cgroup_release_agent_write` / cgroup v1
Weakness Theme	Improper authorization around release_agent handling
Severity	Critical
Attack Vector	Local (often reachable from container context)
Privileges Required	Low / constrained local foothold depending on environment
User Interaction	None
Primary Impact	Privilege escalation and potential container escape
Public Exploitation Signal	Added to CISA KEV
High-Risk Environments	cgroup v1 hosts, exposed container platforms, insufficiently hardened Kubernetes / Docker nodes

3. Source Review & Confidence

Source	Contribution	Confidence
NVD	Normalized vulnerability description and weakness framing	High
CISA KEV	Confirms exploitation priority and remediation urgency	High
Public technical research (e.g., container security writeups)	Provides container-escape and privilege-escalation context	Medium-High

Confidence is high that the flaw supports meaningful privilege-boundary bypass on vulnerable Linux systems, especially where cgroup v1 and permissive container settings coexist. Exploitability in a given environment depends on kernel version, cgroup configuration, namespace behavior, and container runtime hardening.

4. Vulnerability Details

The public description states that unauthorized users can modify the `release\_agent` file at the root of the cgroup hierarchy. In secure configurations, writing a release agent path should be tightly controlled because the release agent can be invoked by the kernel in response to cgroup lifecycle events.

Abuse of this behavior can let an attacker arrange execution of attacker-controlled commands on the host. This is why the flaw is frequently discussed not just as a local privilege-escalation issue, but as a container escape risk in environments where a malicious workload can access the relevant cgroup interfaces.

5. Attack Scenarios

1. Container-to-host escape

- An attacker gains code execution inside a vulnerable Linux container.
- By abusing cgroup v1 release-agent behavior, the attacker escapes the container boundary and executes code on the underlying host.

2. Local privilege escalation on shared Linux infrastructure

- A low-privileged user or compromised service account on a vulnerable host abuses the flaw to gain elevated privileges.
- The attacker pivots to credential theft, lateral movement, or persistence.

3. Cluster foothold expansion

- A compromised Kubernetes or Docker workload uses the vulnerability to take over a worker node.
- The node compromise is then used to access secrets, images, service-account tokens, or neighboring workloads.

6. Threat Landscape & Exploitation Status

CISA KEV inclusion indicates that exploitation has been significant enough to warrant federal remediation tracking. The vulnerability is especially attractive in cloud-native and containerized environments because a successful exploit can convert a constrained application compromise into host-level control.

Defenders should assume that adversaries who target Linux container platforms are familiar with this class of escape and may chain it with weak workload isolation, permissive mounts, or excessive Linux capabilities.

7. Affected Products and Exposure

Product / Surface	Exposure Consideration	Priority
Linux systems using vulnerable kernels	Directly affected until patched	Immediate
Container hosts using cgroup v1	Elevated risk for escape-style abuse	Immediate
Kubernetes / Docker worker nodes with weak hardening	Higher likelihood of practical exploitation	High
Hardened cgroup v2-only environments	Reduced exposure, but validate kernel and runtime posture	Medium

Defenders should inventory Linux kernel versions, determine whether cgroup v1 remains enabled, and identify workloads with capabilities, mounts, or runtime permissions that increase exploit feasibility.

8. Mitigation and Workarounds

4. Patch vulnerable Linux kernels to vendor-fixed versions.
5. Prefer cgroup v2 where supported and operationally feasible.
6. Harden container platforms by minimizing Linux capabilities, preventing privileged containers, and restricting access to sensitive host filesystems.
7. Review Kubernetes pod security controls, seccomp/AppArmor/SELinux policy, and hostPath mount usage.

8. Hunt for unexpected writes to cgroup control files, suspicious child process execution from containers, and anomalous node-level persistence artifacts.

9.1 Detection Opportunities

Signal	Rationale	Data Source
Writes to `release_agent` or unusual cgroup control paths	Closest behavior tied to published exploit mechanics	Auditd, eBPF, file integrity monitoring
Container-launched processes spawning host-level shells or utilities	Common sign of breakout or post-escape activity	EDR, process telemetry
Unexpected privileged pod / container behavior	May reveal exploitation prerequisites or follow-on abuse	Kubernetes audit logs, runtime telemetry
Node-level file or service modifications shortly after container activity	Can indicate successful host compromise	EDR, Sysmon for Linux, auditd

9.2 Detection Query

```
// Hunt for suspicious process behavior originating from containerized workloads on Linux hosts
DeviceProcessEvents
| where Timestamp > ago(14d)
| where InitiatingProcessAccountName has_any ("root", "containerd", "docker", "kubelet")
| where FileName has_any ("sh", "bash", "curl", "wget", "python", "perl", "nc")
| project Timestamp, DeviceName, FileName, ProcessCommandLine, InitiatingProcessFileName,
InitiatingProcessAccountName, FolderPath
| order by Timestamp desc
```

9.3 Exposure / Triage Query

```
// Identify Linux hosts likely involved in container orchestration or runtime activity
DeviceProcessEvents
| where Timestamp > ago(30d)
| where FileName has_any ("dockerd", "containerd", "kubelet", "crio", "runc")
| summarize FirstSeen=min(Timestamp), LastSeen=max(Timestamp), Processes=make_set(FileName, 10) by
DeviceName
| order by LastSeen desc
```

10.1 Hunt Leads

- Hunt Linux audit logs for writes to `release\_agent`, `notify\_on\_release`, or related cgroup v1 control files.
- Review worker nodes for suspicious shells or utilities launched from container runtime parents.
- Investigate pods or containers granted privileged mode, broad capabilities, host PID/IPC access, or risky hostPath mounts.
- Correlate container process ancestry with subsequent node-level file writes, service creation, or outbound callbacks.

10.2 Hunt Query

```
// Pivot from container runtime activity into possible node-level compromise behavior
DeviceFileEvents
| where Timestamp > ago(14d)
| where FolderPath has_any ("/sys/fs/cgroup", "/etc/systemd/", "/usr/lib/systemd/", "/root/", "/tmp/")
| summarize ChangedFiles=count(), SamplePaths=make_set(FolderPath, 20) by DeviceName,
InitiatingProcessFileName, InitiatingProcessAccountName
| order by ChangedFiles desc
```

11. MITRE ATT&CK Mapping

Tactic	Technique	Relevance
Privilege Escalation	T1068 Exploitation for Privilege Escalation	Core impact on vulnerable Linux systems

Escape to Host / Lateral Expansion Context	T1611 Escape to Host	Relevant in containerized environments
Persistence	T1505 / system-level persistence techniques	Likely follow-on behavior after host compromise

12. Validation / Lab Testing

9. Verify kernel versions on all Linux hosts and container nodes against vendor-fixed releases.
10. Confirm whether cgroup v1 remains enabled anywhere in production or staging.
11. Validate that privileged containers, risky host mounts, and excessive capabilities are minimized.
12. Test host and container telemetry to ensure writes to sensitive cgroup paths and runtime-spawned shells are observable.
13. Review node baselines for unexpected persistence or drift if historical exposure is suspected.

13. Recommended Actions Summary

Priority	Action
Immediate	Patch vulnerable Linux kernels
Immediate	Identify and reduce cgroup v1 exposure on container hosts
High	Remove privileged containers and minimize dangerous capabilities / mounts
High	Hunt for suspicious cgroup control-file access and runtime-spawned shell activity
Medium	Strengthen node monitoring and Kubernetes / container runtime hardening

14. References

14. NVD entry for CVE-2022-0492
15. CISA Known Exploited Vulnerabilities catalog entry for CVE-2022-0492
16. Public technical research discussing cgroup v1 release_agent abuse and container escape implications